RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving

Wenqi Jiang*
ETH Zurich
Zurich, Switzerland
wenqi.jiang@inf.ethz.ch

Suvinay Subramanian
Google
Mountain View, CA, USA
suvinay@google.com

Cat Graves
Google DeepMind
Mountain View, CA, USA
catgraves@google.com

Gustavo Alonso
ETH Zurich
Zurich, Switzerland
alonso@inf.ethz.ch

Amir Yazdanbakhsh†
Google DeepMind
Mountain View, CA, USA
ayazdan@google.com

Vidushi Dadu†
Google
Sunnyvale, USA
vidushid@google.com

Abstract

Retrieval-augmented generation (RAG) is emerging as a popular approach for reliable LLM serving. However, efficient RAG serving remains an open challenge due to the rapid emergence of many RAG variants and the substantial differences in workload characteristics across them. This paper makes three fundamental contributions to advancing RAG serving. First, we introduce **RAGSchema**, a structured abstraction that captures the wide range of RAG algorithms, serving as a foundation for performance optimization. Second, we analyze several representative RAG workloads with distinct RAGSchema, revealing significant performance variability across these workloads. Third, to address this variability and meet diverse performance requirements, we propose **RAGO** (Retrieval-Augmented Generation Optimizer), a system optimization framework for efficient RAG serving. RAGO achieves up to a 2× increase in QPS per chip and a 55% reduction in time-to-first-token latency compared to RAG systems built on LLM-system extensions.

CCS Concepts

• **Information systems** → **Information retrieval**; • **Computer systems organization** → **Architectures**; • **Computing methodologies** → **Natural language processing**; • **Hardware** → **Emerging simulation**.

Keywords

Retrieval-Augmented Generation, Computer System, Computer Architecture, Large Language Model, Performance Optimization

ACM Reference Format:
Wenqi Jiang, Suvinay Subramanian, Cat Graves, Gustavo Alonso, Amir Yazdanbakhsh, and Vidushi Dadu. 2025. RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3695053.3731093

*Work done while at Google.
†Equal advising.

Artifact: https://github.com/google/rago.

This work is licensed under a Creative Commons Attribution 4.0 International License.

ISCA '25, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731093

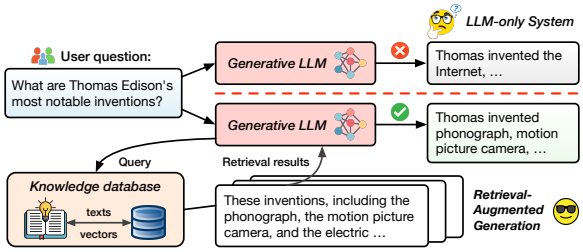


Figure 1: LLM-only system (top) versus RAG (bottom).

1 Introduction

The rapid adoption of Large Language Models (LLMs) across diverse applications — such as question answering [23, 26, 29], code generation [73, 76, 90], and scientific discoveries [20, 67, 107] — showcases their profound impact on automating knowledge-based tasks. Despite these capabilities, LLM systems, when deployed in isolation (aka LLM-only systems), face substantial challenges, such as data staleness [63, 71], a propensity to hallucinate (generating factually incorrect or nonsensical information), and limited, often rigid model knowledge [46, 71, 74]. These challenges hinder the reliability and adaptability of LLM-only systems, especially in applications that demand high factual accuracy [31, 42, 57, 109].

Retrieval-Augmented Generation (RAG) has emerged as one powerful solution to address the common pitfalls of LLM-only systems for knowledge-intensive tasks [22, 63, 70, 99, 112]. By retrieving information from external databases and appending it to prompts (Figure 1), RAG enhances the credibility, timeliness, and contextually rich nature of LLM-generated responses. Leveraging the generative prowess of LLMs alongside external knowledge sources, RAG not only achieves comparable quality with smaller models [22, 99, 112] but also simplifies the process of updating knowledge, mitigating the extent of additional model training, which is often prohibitively expensive [63, 70]. These advantages have established RAG as the industry standard for knowledge-intensive applications, with notable examples including Google's REALM [39] and RETRO [22], Meta's MARGE [70], Microsoft's GraphRAG [30], and NVIDIA's InstructRETRO [112]. As companies race to integrate RAG systems into their production pipelines [28, 84, 97], optimizing their performance has become increasingly critical.

In contrast to conventional LLM-only serving systems, which center predominantly on optimizing the prefix (prompt decoding) and decoding (token generation) stages, RAG presents three challenges: **(C1)** RAG systems are intrinsically heterogeneous, comprising a diverse array of system components, including vector search-based retrieval [22, 71, 99], generative LLMs [23, 84, 108], and multiple optional models such as database encoders [65, 96], query rewriters [24, 79], and retrieval result rerankers [13, 35]. These components often run on heterogeneous hardware platforms. For example, retrievals are typically performed on CPU servers, whereas ML accelerators (e.g., TPUs or GPUs) are used for model serving. This interplay of diverse components and hardware platforms amplifies the search space, far surpassing that of LLM-only systems; **(C2)** Various RAG configurations defined by factors such as database size, retrieval frequency, model selection, and serving hardware, exhibit substantial performance variability. This variability can veer the bottleneck between inference and retrieval or among different models within the serving pipeline; and **(C3)** A natural consequence of the heterogeneity in components and the variability in performance is the emergence of a new challenge: *how can we design efficient RAG serving systems?* Addressing this challenge demands meticulously navigating key decisions in scheduling policies across diverse RAG configurations and hardware platforms.

To address these challenges in optimizing RAG serving performance, we ground our approach in three key design principles: (1) **Workload abstraction**: Tackling the heterogeneity of RAG systems necessitates an abstraction to encapsulate the diverse RAG workloads. Without such abstraction, the inherent complexity of RAG configurations become intractable; (2) **Critical system design decisions**: To unveil the critical system design decisions and illuminate the performance trade-offs inherent in RAG serving, a careful performance characterization of representative RAG workloads is warranted. Without understanding how these different workloads behave, the optimization process risks becoming guesswork; and (3) **Systematic optimization framework**: To navigate the large optimization space arising from the Cartesian product of RAG workload and system design dimensions, an optimization framework is essential to uncover and exploit efficiency opportunities in RAG serving systems.

To systematically describe RAG workloads, we introduce **`RAGSchema`** (§3), a RAG serving abstraction that encapsulates a set of essential performance-relevant workload attributes. `RAGSchema` includes two key components: (a) specification of the RAG pipeline —document encoder, query rewriter, result reranker, and generative LLM— and (b) model and retrieval configurations, including model size, database size, the number of query vectors per retrieval, and iterative retrieval frequency. This abstraction simplifies the representation of complex RAG workloads while providing sufficient information for performance characterization and optimization.

Building on `RAGSchema`, we perform a detailed workload characterization (§5) to identify bottlenecks and key system design decisions. We analyze four representative RAG paradigms, each with distinct RAG pipeline: (a) RAG with hyperscale retrieval [22, 99, 112]; (b) RAG for long-context sequence processing [64, 75, 122]; (c) RAG with iterative retrieval [22, 52, 110]; and (d) RAG with query rewriter and retrieval reranker models [13, 24, 35, 79]. Our analysis reveal *significant performance variability both across and within paradigms,*

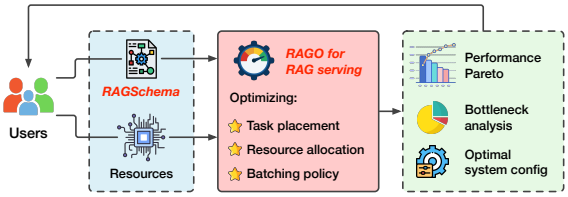


Figure 2: `RAGO` for systematic RAG serving optimization.

with a subset of findings summarized as follows. First, bottlenecks shift between retrieval and inference across RAG paradigms. For instance, hyperscale retrieval can spend over 80% in retrieval (§5.1) while in long-context scenarios, retrieval accounts for less than 1% of the total latency (§5.2). Second, even smaller models within the pipeline can significantly influence system performance. For example, in long-context processing, a database encoder that is 100× smaller than the main generative LLM can become the bottleneck due to the large number of tokens it must process (§5.2). Third, iterative retrievals during decoding can stall the pipeline, as the decoding process waits for retrieval results (§5.3). The insights from these studies underscore not only the importance of making appropriate system design decisions but also the indispensable need for a tailored optimization framework for RAG serving systems, given their far less predictable performance landscape compared to LLM-only systems.

To this end, we introduce **`RAGO`** (Retrieval-Augmented Generation Optimizer), a system performance optimization framework for efficient RAG serving (Figure 2). Given a RAG workload represented by `RAGSchema` and system resource constraints, this framework explores the scheduling policy space to determine optimal schedules aligned with user-defined performance objectives. Key scheduling decisions of `RAGO` include deciding whether inference components are collocated or disaggregated across ML accelerators (*task placement*), assigning the type and quantity of resources to each component (*resource allocation*), and tuning batch sizes for retrieval and inference tasks to balance throughput and latency (*batching policies*). `RAGO` uses an analytical cost model, inspired by [40, 88, 125], to identify the performance Pareto frontier and generate corresponding system schedules. This cost model is based on XPU, a generic systolic-array ML accelerator [6, 56, 125], and serve as the core engine of `RAGO` for evaluating various RAG paradigms and configurations. Below, we summarize the key contributions of our work:

- We propose `RAGSchema`, a RAG workload abstraction that simplifies RAG workload representation and enables systematic performance characterization and optimization.
- Using `RAGSchema`, we identify key system design decisions and performance trade-offs from characterizing four representative RAG paradigms and their instantiations.
- We develop `RAGO`, a systematic optimization framework that optimizes scheduling policies for efficient RAG serving. Our results show that `RAGO` delivers up to 2× improvement in QPS per chip and a 55% reduction in time-to-first-token latency compared to RAG serving systems built on LLM-only systems.

2 Background

Why RAG outshines LLM-only systems? LLM-only systems often struggle to achieve high factual accuracy or providing up-to-date information. RAG addresses these limitations by combining

the linguistic capabilities of LLMs with real-time knowledge retrieval [22, 63, 70, 95, 99, 112, 116, 118, 121]. During offline preprocessing, external textual knowledge is encoded as vectors using an LLM and stored in a vector database. At serving time, relevant knowledge is retrieved via vector search, assessing relevance by comparing the similarity between prompt's vector representation and those in the database. The retrieved knowledge is then appended to the prompt, refining the quality of the LLM's responses.

By virtue of this combination, RAG systems offer several key advantages over LLM-only systems for knowledge-intensive tasks. *First*, RAG systems simplify knowledge updates by allowing external databases to be modified independently [22], unlike LLMs, which require retraining or fine-tuning [22, 71]. *Second*, RAG reduces "hallucination"—a phenomenon where LLMs generate factually incorrect or entirely fictitious information. For instance, a traditional LLM might confidently assert that "Thomas Edison invented the internet", despite this being obviously false. RAG's reliance on external, up-to-date databases helps mitigate these errors by grounding the model's output in real, retrievable data [71, 74]. *Finally*, RAG systems achieve comparable or better generation quality with models that are one to two orders of magnitude smaller than LLMs [38, 42, 59–61, 70, 71]. While conventional LLMs require extensive parameters to encode a vast range of general knowledge [23, 26, 92, 103], RAG partially offloads this knowledge storage to an external database, retrieving only the most relevant content during inference.

LLM-only serving systems. Serving LLM-only systems typically involves two distinct stages: prefix (prompt computation) and decode (token generation) [89, 132]. The prefix stage processes the input prompt to generate the first output token and populate the associated key-value (KV) cache [111], which holds the encoded representation of the input context. The decode stage, on the other hand, generates subsequent tokens one at a time in an auto-regressive manner, relying on the KV cache from the prefix stage.

Modern LLM serving systems [89, 132] often disaggregate these stages, running them on separate accelerators to accommodate their distinct workload characteristics, similar to disaggregated designs in recommender systems [47, 58]. The prefix stage processes the entire input sequence at once, making it highly compute-intensive. Even with small batches, the prefix stage benefits from accelerators with high computational throughput to handle the full sequence length efficiently [89]. In contrast, the decode stage is memory-bound, as each inference step requires accessing the KV cache of previous tokens, while the amount of computation is small [89]. In addition to workload differences, these two phases affect different performance metrics with different SLAs: time-to-first-token (TTFT) for the prefix phase and time-per-output-token (TPOT) for the decode phase. Ultimately, optimizing the performance of LLM-only serving often depends on efficient resource allocation between the prefix and decode stages [89].

Vector search for retrieval. Another core component in RAG systems is retrieval, which identifies information from external knowledge databases. A common approach to performing this retrieval is *vector search*, which has become the cornerstone of recent information retrieval systems [45, 83]. Vector search enables the system to assess semantic relevance by encoding both documents and queries as high-dimensional vectors (e.g., hundreds to thousands dimensions), where proximity in this vector space reflects semantic similarity.

In practice, vector search retrieves the $\mathcal{K}$ most similar vectors to a given $\mathcal{D}$-dimensional query vector x from a database $\mathcal{Y}$ populated with many $\mathcal{D}$-dimensional vectors. This similarity is typically computed using metrics such as L2 distance or cosine similarity [45, 83]. Since exact $\mathcal{K}$ Nearest Neighbor (KNN) search is costly on large-scale datasets, real-world vector search systems adopt *Approximate Nearest Neighbor* (ANN) search algorithms, which provide a scalable alternative to exact KNN by trading recall for much higher system performance.[1]

The *IVF-PQ* algorithm, which combines an inverted file (IVF) index with product quantization (PQ) [45], is one of the most widely used approaches for large-scale vector search in RAG [22, 42, 60]. IVF-PQ is frequently preferred over other ANN algorithms, such as graph-based search algorithms [32, 33, 78, 82, 83, 130, 133], due to its memory efficiency (e.g., one byte can represent 4∼16 dimensions in PQ [45, 50, 54]) —a crucial advantage when RAG systems operate on large databases, sometimes containing up to 64 billion vectors (92 TB before quantization) [22, 112].

Two popular open-source libraries for IVF-PQ are Faiss [7] and ScaNN [10, 37], exemplifying CPU-bound and memory-bound PQ variants, respectively. Faiss employs a high-precision quantization variant, resulting in a CPU-bound search process [14, 50]. In contrast, ScaNN adopts lower-precision quantization [106], achieving higher CPU throughput and shifting the workload toward being memory-bound.

3 Structuring the Complex RAG Serving Terrain

In this section, we first describe four representative RAG paradigms with increasingly diverse and complex RAG pipelines. We then describe `RAGSchema` (§3.2), a structured abstraction to capture this workload diversity, serving as a foundation for serving performance characterization (§5) and optimization (§6).

3.1 Representative RAG Paradigms

We now show the workload diversity by describing the following representative RAG paradigms:

Paradigm I: Hyperscale Retrieval. Retrieval over a large-scale corpus combined with smaller LLMs can serve as an alternative of larger LLMs without retrieval [22, 99, 112]. Prior work has shown that RAG systems can match or even surpass the quality of LLM-only systems when database sizes are sufficiently large [22, 99]. This is achieved while using sufficiently smaller models —approximately one-tenth the parameters of their LLM-only counterparts [22, 112]. This quality parity is achieved because LLM-only models rely on their vast parameter sets to encode comprehensive knowledge during training [23, 26, 92, 103], whereas RAG systems dynamically integrate external knowledge at inference time, reducing the need for extensive parameterization within the model itself.

Paradigm II: Long-Context Sequence Processing. Another common paradigm is to use RAGs to facilitate long-context processing [64, 75, 122]. For example, when answering questions based on a lengthy document (e.g., with more than 100K tokens) that a user

[1]We use "vector search" and "ANN search" interchangeably.

Table 1: RAGSchema component names, attributes, and corresponding example design parameters.

RAGSchema Components	Attributes	Examples
Document Encoder	Model size (parameters) of the encoder used to convert database documents and queries into vector representations.	120M
Vector Dimensionality	The number of dimensions for each database vector.	768-dim
Database Vector Number	Number of the database vectors, depends on the corpus size and passage chunk lengths.	1,000
Retrieval Frequency	Whether iterative retrievals are permitted during decoding and number of retrievals per sequence.	Four per sequence
Queries Per Retrieval	Number of query vectors used per retrieval (one or multiple).	Two per retrieval
Query Rewriter	Model size of the generative query rewriter, if applied.	8B
Query Reranker	Model size of the retrieval results reranker (usually an encoder-only model), if applied.	120M
Generative LLM	Represents the model size of the main generative LLM used for answer generation.	70B

has uploaded in real time — similar to use cases in Gemini 1.5 [108], NotebookLM [8], and ChatGPT [9] — a straightforward approach is to include the entire document in the prompt. However, this approach is often prohibitively expensive due to the large number of tokens to process. Instead, an efficient alternative is to treat the user-provided long document as a knowledge database, retrieving only the relevant information needed to answer the questions. This method substantially reduces the prompt size by avoiding the need to load the full text into the model's context window. Recent studies [64, 122] demonstrate that this retrieval-based approach achieves similar response quality to using the full document as a prompt, providing a practical balance between cost and quality in handling long contexts. In contrast to the paradigm I, RAG for long-context processing introduces two key modifications. First, this setup includes a database encoder, which is necessary for constructing the database when the long context is initially provided. Second, the database is orders of magnitude smaller. For example, given a context length of 100K tokens and a passage chunk size of 100 tokens, the database only consists of 1K vectors, compared to tens to hundreds of billions of vectors in paradigm I [22, 99].

Paradigm III: Iterative Retrievals. While a single retrieval at the beginning may suffice in some scenarios, recent studies [22, 52, 110] indicate that iterative retrievals —periodically updating retrieved content during generation— can significantly enhance model quality. Such update of the retrieved content is particularly valuable in scenarios requiring multi-hop reasoning, where each retrieval provides additional context to guide the subsequent token generation process [110, 122]. In this configuration, the decoder initiates retrievals at flexible intervals during generation. Upon issuing a retrieval, the generation of this sequence temporarily pauses the token generation, to process newly retrieved content through the prefix phase. Only after integrating this additional context does the decoder continue generating the rest of sequence.

Paradigm IV: Query Rewriter and Reranker. Users often pose vague or complex queries, making it challenging to retrieve relevant information directly. To address this, the retrieval process can be significantly improved by incorporating pre-processing and post-processing steps [13, 24, 35, 79]. For pre-processing, recent studies [24, 79] demonstrate that leveraging an LLM to rewrite the user's query can improve retrieval quality. This LLM may either rephrase the query for clarity or decompose complex questions into multiple simpler queries that cover different aspects of the user's original intent [24, 79]. Once the initial results are retrieved through vector search, a reranking model can be applied as a post-processing step [4, 13, 35]. The reranker improves content retrieval quality by scoring each document's relevance beyond simple vector similarity and choosing documents that more closely align with the user's intended question.

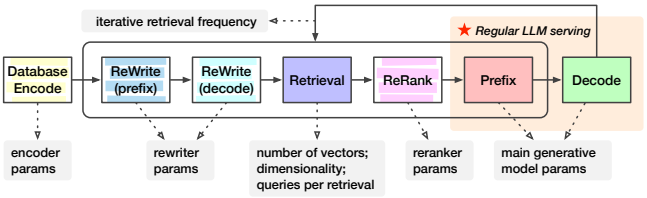


Figure 3: Describing general RAG pipelines with RAGSchema.

3.2 RAGSchema for Workload Abstraction

Given these diverse paradigms, RAG workloads exhibit significant variability across algorithm configurations in the following ways. First, retrieval configurations can vary dramatically. Database sizes may span several orders of magnitude (e.g., a million times) [22, 64, 75, 99]; a retrieval may involve not a single query vector [39, 70] but multiple ones [21, 24, 113]; and some models support iterative retrievals during the generation process [22, 52, 110]. Second, a RAG system may include several models in addition to the main generative LLM. These auxiliary models include a database encoder for processing real-time uploaded documents [65, 96]; a query rewriter model [24, 79] to rephrase user queries; and a result reranker model [4, 13, 35] to score retrieved information.

To navigate the complex RAG configuration space, we introduce **RAGSchema**: *a structured and modular abstraction that captures the key performance-relevant attributes of various RAG serving workloads.* As visualized in Figure 3 and detailed in Table 1, RAGSchema defines both (1) the execution flow of the RAG pipeline and (2) the configuration of its components. For the RAG pipeline definition, optional stages[2]—such as the database encoder, query rewriter, reranker, and iterative retrieval— can be included or omitted. For each included component, RAGSchema specifies relevant configurations, including model parameter counts, vector dimensionality, number of database vectors, queries per vector, and iterative retrieval frequency if applicable. While RAGSchema abstracts RAG serving workloads, it is not an abstraction for quality, as different models and databases of the same size can lead to varying quality.

3.3 Empirical RAG Performance Analysis

Even though precise bottlenecks and tradeoffs depend on exact RAGSchema, high-level performance bottlenecks in RAG systems are driven by RAG workload pipelines and the Amdahl's law. In this

[2]A "stage" refers to the execution of a RAG pipeline component.

section, we make general observations about RAG workloads, and quantify in §5 using detailed performance models.

We represent inference throughput as a function of FLOPs, and retrieval throughput as a function of the bytes of database vectors accessed. Note the precise throughput depends on CPU server efficiency, accelerator capability, scheduling policies, etc.

Inference components. For a model with size M and a sequence length L, the FLOPs required for processing the entire sequence are approximately: $\text{FLOPs}_{\text{inference}} \approx 2 \cdot M \cdot L$ for short sequences (e.g., $L \leqslant 10^3$) where the quadratic complexity of the attention mechanism still has negligible impact.

Retrieval component. The retrieval workload can be approximately described by the number of bytes of database vectors processed per query. Unlike model inference, decoding quantized database vectors represents a fundamentally different workload where FLOPs is not an appropriate metric [14, 50]. Given a database with N_{dbvec} vectors, where each vector consists of B_{vec} bytes, and each query scans a subset of P_{scan} percent of database vectors, the total bytes to scan per query is approximately: $\text{B}_{\text{retrieval}} \approx N_{\text{dbvec}} \cdot B_{\text{vec}} \cdot \frac{P_{\text{scan}}}{100}$. Here, P_{scan} is determined by evaluating a set of sample queries and analyzing the relationship between P_{scan} and retrieval quality measured by recall, as a common practice for retrieval configuration tuning [5]. The minimum value of P_{scan} that satisfies the required retrieval quality is then selected.

End-to-end RAG performance. While the latency of RAG serving is the sum of the latencies of each stage in the RAG pipeline, the throughput of the pipeline is determined by its slowest stage (excluding iterative retrieval paradigm for now, as it follows a different pattern discussed in §5.3). For a RAG pipeline with m stages, where each stage has a throughput denoted by QPS_i ($i = 1,2,\ldots,m$), the end-to-end RAG serving throughput is: $\text{QPS}_{\text{RAG}} = \max(\text{QPS}_1, \text{QPS}_2, \ldots, \text{QPS}_m)$.

From this high-level model, we can draw several key insights. First, retrieval can become a bottleneck when its workload ($N_{\text{dbvec}} \cdot B_{\text{vec}} \cdot \frac{P_{\text{scan}}}{100}$) is high while the inference workload ($2 \cdot M \cdot L$) is relatively low. Second, in paradigms with multiple inference components, any model can become critical depending on its size M and processed sequence length L, which may vary based on the model's role. Finally, the cumulative effect of multiple inference stages and retrievals can significantly impact overall serving performance. We discuss detailed evaluation methodology and quantitative characterization in the subsequent sections.

4 Methodology

This section outlines the methodology used to characterize RAG performance (§5) and evaluate RAGO across various configurations (§7).

Models and database. We evaluate four LLMs—Llama-3 1B, 8B, 70B, and 405B [29]—covering size scales comparable to those used in [22, 99, 112]. We assume the models are quantized to 8-bit integer, thus the accelerator memory requirement directly corresponds to the model's parameter count (e.g., a 70B model requires 70 GB of memory). As RAG quality continues to benefit from larger knowledge corpora [22, 99], we adopt a hyperscale database [22]. This database contains 64 billion passages, each encoded as a 768-dimensional vector [22], making it approximately 400× larger than the largest academic vector search datasets [12, 16, 102]. We apply product quantization (PQ) as in [22] to compress each vector to 96 bytes (1 byte per 8 dimensions), resulting in a 5.6 TiB quantized vector database. Following the index recommendations of the ScaNN library [10], we use a balanced fanout of 4K vectors per node across the three-level tree index [105] ($(64 \times 10^9)^{1/3} = 4 \times 10^3$). To balance retrieval quality and performance, each query is compared against 0.1% of the database vectors by default, as this setup has shown high recall (over 90%) in billion-scale datasets [50].

Table 2: Performance specifications of three versions of XPUs. We report performance on XPU-C (⋆) by default.

	XPU-A	XPU-B	⋆ XPU-C
TFLOPS	197	275	459
HBM (GB)	16	32	96
Mem. BW (GB/s)	819	1200	2765
Inter-Chip Link BW (GB/s)	200	300	600
Resembles	TPU v5e [2]	TPU v4 [1]	TPU v5p [3]

LLM sequence lengths. In line with common RAG use cases such as question-answering [24, 70, 110], we evaluate sequence lengths derived from QA datasets [17, 55, 94], where the question lengths range from six to 42 tokens. To simplify the search space, we use 32 tokens as the typical question length. The input prompt length includes both the question and relevant retrieved content. The typical nearest neighbor retrieved ranges from two to 10 [15, 22, 52, 110], each with an average length of 100 tokens. We pick five as a common value for our evaluations. Given this, we approximate the average length of input prompt (question + relevant retrieved contents) to 512 tokens. For generation lengths (decode stage), we rely on data from long-form QA [31] and chatbot datasets [11, 62], selecting 256 tokens as a representative decode length.

System setup. Our evaluation assumes a data center model serving environment with abundant resources to support various system configurations. Across the RAG serving stages (e.g., prefix, decode), we allocate a total of 16 to 32 servers hosting 64 to 128 XPUs (4 XPUs per server), as a minimum of 16 servers is required to ensure sufficient host memory capacity for the dataset (5.6 TiB after quantization). An XPU refers to a generic systolic-array-based ML accelerator [6, 56]. The number of XPUs allocated to each model component is configured in powers-of-two scaling factors (e.g., 1, 2, 4, etc.). Each XPU, inspired by the setup of TPU v5p accelerators [3], is equipped with 96 GB of high-bandwidth memory (2.7 TB/s) and 459 TFLOPS of compute capacity. The XPUs are interconnected via a high-bandwidth 3D torus topology, offering 600 GB/s of inter-chip bandwidth (six 100 GB/s links per chip). We also evaluate two other versions of XPUs, as shown in Table 2, for ablation studies. The host CPUs are modeled after AMD EPYC Milan processors, featuring 96 cores, 384 GB of memory, and 460 GB/s of memory bandwidth. We assume that XPU host servers support distributed retrieval across large databases.

Simulation setup. RAG performance is reported by assembling the costs of all model inference and retrieval stages, based on a search across various system configurations (details described in §6). We now describe the production-grade simulators used to measure inference and retrieval performance. With the following simulation mechanism, we can model RAG serving performance given XPUs and CPU servers with varying configurations, as well as different distributed serving configurations in data centers.

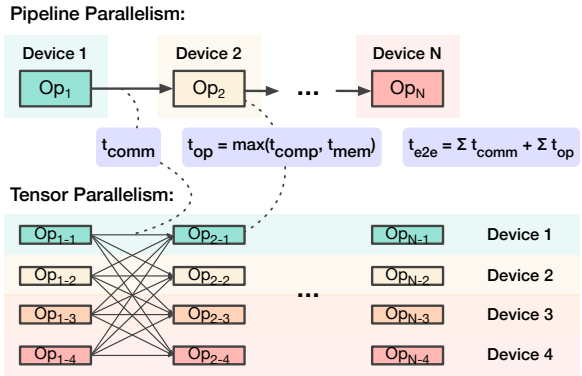


Figure 4: Parallelisms and cost model in model inference.

(a) Inference performance modeling. We adopt an in-house calibrated XPU simulator for inference simulation. The simulator is well-correlated with the production-grade XPU accelerators across a set of real-world ML models. For multi-XPU inference, the simulator evaluates a range of model sharding strategies, where each accelerator unit is assigned a subset of inference operators. The simulator supports pipeline parallelism [41, 86], tensor parallelism [93, 101], and hybrid approaches. As shown in Figure 4, the simulator abstracts inference as a sequence of operators, following a methodology similar to other established ML simulators [88, 125]. That is, the total latency is computed as the sum of each operator's execution time and the associated communication costs between operators. The execution time of each operator is calculated using a roofline model, where latency is determined by the maximum of memory access latency and compute latency: $T_{op_i} = \max\left(\frac{F_i}{P_{\text{comp}}(F_i)}, \frac{D_i}{B_{\text{mem}}(D_i)}\right)$, where F_i is the number of floating-point operations required by operator op_i, P_{comp} represents the compute performance, D_i denotes the total data size processed by op_i, and B_{mem} is the available memory bandwidth. Additionally, the communication latency between two operators depends on the volume of data transferred and the network bandwidth: $T_{comm}(op_i, op_j) = \frac{S_{i,j}}{B_{\text{net}}}$, where $S_{i,j}$ represents the size of data transferred between op_i and op_j, and B_{net} is the available network bandwidth (bytes/second).

(b) Retrieval performance modeling. Our retrieval simulation is based on ScaNN [10, 37], a product quantization library (§2) that demonstrates state-of-the-art performance across dozens of algorithms in the ANN benchmark [5]. We implement the ScaNN performance model described in [105], which models the search process as a sequence of vector scan operations at each level of a multi-level tree [10, 105]. The total retrieval latency is calculated as the sum of the latencies for these scan operations. ScaNN dedicates one thread per query and parallelizes batches of queries across multiple threads. The cost of each operator is calculated by a roofline model that factors in batch sizes, the number of CPU cores, per-core CPU processing throughput, and memory bandwidth. The execution time of each scan operator is determined by the maximum of compute time and memory access time: $T_{op_i} = \max\left(\frac{D_i}{P_{\text{comp}}(Q)}, \frac{D_i}{B_{\text{mem}}(D_i)}\right)$, where D_i represents the total number of bytes processed by operator op_i, Q denotes the query batch size, P_{comp} is the CPU throughput for handling the scan operation, and B_{mem} represents the memory bandwidth.

Table 3: `RAGSchema` of the workloads used in case studies.

`RAGSchema` Components	Case 1	Case 2	Case 3	Case 4
Document Encoder	N/A	120M (768-d)	N/A	N/A
Database Vector Number	64B	1/10/100K	64B	64B
Retrieval Frequency	1	1	2/4/8	1
Queries Per Retrieval	1/2/4/8	1	1	1
Query Rewriter	N/A	N/A	N/A	8B
Query Reranker	N/A	N/A	N/A	120M
Generative LLM	1/8/70/405B	8/70B	8/70B	8/70B

For large databases requiring distributed search across multiple servers, we assume each server holds a shard of the dataset with independent indexes. Queries are routed to all servers, and results are aggregated. The workload is balanced across servers, with negligible overhead for broadcast and gather operations.

To populate simulator parameters, we benchmark the maximum achievable per-core throughput and memory bandwidth by running open-source ScaNN [10] on smaller datasets configured with the same tree node sizes (4K vectors per node) as the 64-billion vector database. On AMD EPYC 7R13 CPUs with 24 cores, ScaNN achieved a PQ code scanning throughput of 18 GB/s per CPU core, with approximately 80% memory bandwidth utilization. We then calibrate the retrieval performance model using internal production datasets comparable in scale to the 64-billion vector dataset used in our study (§4).

(c) Communication between retrieval and inference. Retrieved documents are transferred from CPUs to XPUs, with each retrieval's data size modeled as $N_{\text{token}} \times B_{\text{token}}$, where N_{token} represents the number of tokens and B_{token} denotes the bytes per token. In practice, this communication overhead is negligible. For instance, retrieving five documents, each containing 100 tokens at 2 bytes per token, results in only 1 KB of data transmission per query. Given a typical PCIe bandwidth of tens of GB/s, a system can support tens of millions of requests per second — several orders of magnitude higher than the typical LLM inference throughput of up to tens of requests per second per XPU, as shown in Figure 5.

Performance metrics. We report common metrics used in the evaluation of LLM systems [89, 132]:

- **[TTFT] Time-to-First-Token** ↦ average latency from request reception to the generation of the first output token.
- **[TPOT] Time-Per-Output-Token** ↦ average latency between generating each output token in a sequence.
- **[QPS] Queries-Per-Second** ↦ maximum throughput, or the number of requests the system can process per second. [3]
- **[QPS/Chip] Queries-Per-Second/Chip** ↦ QPS normalized by chip count, reflecting system cost efficiency.

Since continuous batching [62, 120] is enabled in the decode stage, we report the worst-case TPOT latency. This is because sequences in the batch can be at different stages of generation — some generating the first token and others generating the last ones — and performance is determined by the latter. In contrast, prefix operates deterministically, allowing us to report the precise TTFT latency.

5 RAG Serving Performance Characterization

In this section, we characterize workloads using four case studies, each representing a `RAGSchema` instantiation of a distinct RAG paradigm described in §3.1. These case studies highlight the performance

[3] Note that the term "queries" here does not refer to retrieval queries, and we use the QPS metric exclusively for end-to-end RAG serving performance in this paper.

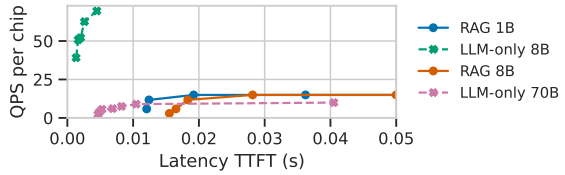


Figure 5: Larger LLM versus RAG with smaller models.

variability across RAG workloads, quantify the impact of paradigm choices on system performance, and motivate the need for our RAG optimization framework (§6). While arbitrary RAG configurations can be constructed beyond these studies, they can be seen as interpolations of the provided cases. For example, a RAG system with a small private database can be seen as a hybrid of Case I and II, where the large-scale retrieval in Case I is replaced with the small-scale retrieval of Case II. Similarly, long-context processing with iterative retrieval can be viewed as a combination of Case II and III.

Characterization methodology. We evaluate performance using the methodology outlined in §4. Unless otherwise specified, the end-to-end performance plots depict the performance Pareto across all system scheduling options. The time breakdown plots are normalized by the resource usage of each component, reflecting time × resource consumption. These plots assume (a) four XPUs per host server and (b) each component operating at its maximum QPS/Chip. Thus, if a plot shows that retrieval time exceeds the combined time of all inference components, the host servers for retrievals are the bottleneck, leaving XPUs occasionally idle; conversely, if inference dominates, retrieval resources may be underutilized. The evaluated workloads are summarized in Table 3. While all configurations in the table are analyzed, the plots highlight a representative subset to avoid redundancy when similar trends are observed across multiple configurations (e.g., model sizes).

5.1 Case I: Hyperscale Retrieval

As introduced in §4, we adopt configurations similar to those in RETRO [22], replicating its database setup and similar sized LLMs along with more recent, larger models. The RAG system performs only *one* retrieval at the beginning, which may involve one or multiple query vectors, as suggested by recent studies [21, 24, 113].

Takeaways: Hyperscale retrieval can pose a significant bottleneck in RAG pipelines. This bottleneck becomes increasingly dominant with (1) smaller LLM, (2) multi-query retrievals, (3) better inference accelerators, (4) shorter prefix and decode sequence lengths, and (5) higher retrieval quality.

System performance comparison (RAG vs. LLM-only). Figure 5 compares RAG and LLM-only systems across different model sizes, with TTFT latency on the x-axis and QPS/Chip on the y-axis. As shown in the RETRO paper [22], RAG can achieve similar or superior generation quality to LLM-only systems with an order of magnitude fewer parameters. Here, we extend this comparison to system performance. Our results indicate that RAG 8B outperforms LLM-only 70B in QPS/Chip by a factor of 1.5×. Although the model size is reduced by approximately 10×, the benefits of using smaller models in RAG are moderated by the retrieval overhead and the need for longer prompts to integrate retrieved information (512 tokens in RAG versus 32-token questions in LLM-only systems), resulting

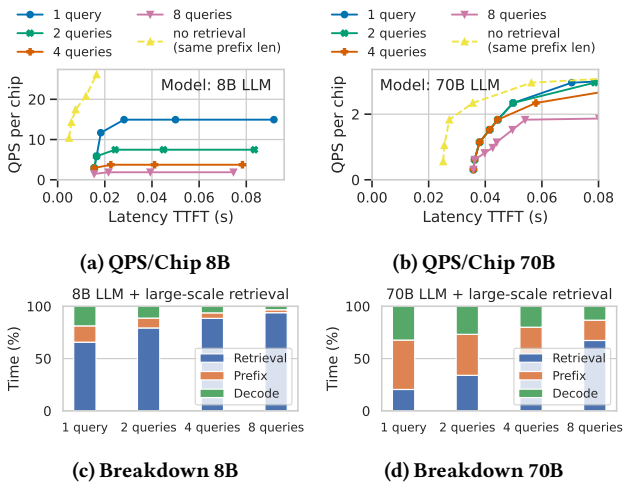


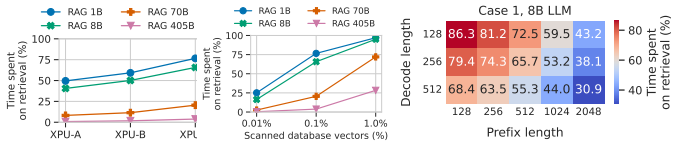
Figure 6: RAG performance given various model size and query numbers for hyperscale retrieval.

in only a 3.2× reduction in inference FLOPs. Consequently, the QPS/Chip gain is not directly proportional to the reduction in parameter size. Interestingly, the results suggest that RAG model sizes can be increased up to a certain limit without compromising QPS/Chip, as retrieval performance is the limiting factor. For example, RAG 1B and RAG 8B exhibit similar QPS, highlighting the importance of system performance analysis in determining how much larger RAG models can scale. While RAG models offer significant advantages at certain scales, their benefits may diminish at lower parameter counts as retrieval latency becomes a bottleneck. For example, despite RAG 1B having only one-eighth the parameters of LLM-only 8B, its QPS/Chip does not scale proportionally, because the retrieval overhead in RAG outweigh the benefits of reduced model size.

Sensitivity to model size. Figure 6a and Figure 6b present the QPS/Chip for the 8B (left) and 70B (right) models, alongside time breakdowns for retrieval, prefix, and decode stages in Figure 6c and Figure 6d. The yellow line represents a "no retrieval" configuration, where retrieval is omitted while the prefix remains the same length. For the 8B model, retrieval is the primary bottleneck; as query counts double, QPS nearly halves due to increased retrieval demands. Conversely, for the 70B model, inference initially limits performance until four queries per retrieval. At higher query vector counts per retrieval (e.g., 8 queries), the bottleneck shifts, and retrieval starts to dominate, as seen in the time breakdown in Figure 6d.

Sensitivity to XPU versions. Figure 7a shows the impact of XPU capability on the percentage of time spent on retrieval for LLMs ranging from 1B to 405B parameters. As the XPU capabilities advance (from version A to C), the proportion of time spent on retrieval increases by up to 25%. While for larger models (e.g. 405B), LLM remains the dominant bottleneck in RAG serving, retrieval is dominant factor for RAG with small models (50% - 75% across XPUs versions). Overall, with more advanced ML accelerators, system efficiency increasingly depends on optimizing retrieval processes.

Sensitivity to sequence lengths. Figure 7c illustrates the sensitivity of retrieval overhead to changes in decode length and prefix length given for 8B model. The retrieval overhead varies



(a) XPU Gen (b) Retrieval Config (c) Seq. Length

Figure 7: The percentage of retrieval time across hardware, retrieval configurations, and sequence lengths in Case I.

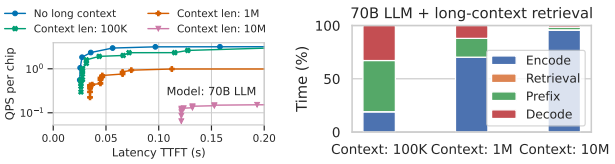
significantly with both decode and prefix lengths — retrieval bottlenecks diminish as sequence lengths increase, shifting retrieval from a primary bottleneck to a secondary factor. For example, 86.3% of the time is spent on retrieval at shorter sequence lengths (e.g., 128 or 256), while the retrieval overhead drops to just 30.9% with longer prefix and decode lengths (2048 and 512). Adjusting the prefix and decode lengths results in unequal changes in the percentage of retrieval time. For example, in a setting of 128 tokens for both prefix and decode, increasing the prefix length to 256 tokens reduces retrieval time from 86.3% to 81.2%, while increasing the decode length to 256 tokens lowers it to 79.4%. This difference occurs because prefix inference is inherently faster than decoding the same number of tokens due to the autoregressive nature of decoding.

Sensitivity to retrieval configurations. Retrieval performance in RAG workflow is highly sensitive to the percentage of database vectors scanned per search. Regardless of the ANN algorithm used, ANN search does not conform to a fixed workload — there is an fundamental trade-off between retrieval performance and quality: scanning more vectors improves quality but reduces performance [45, 83]. This trade-off is further influenced by data distribution; for instance, with the same algorithm, hardware, and QPS, one dataset may achieve over 90% recall, while another may fall below 50% [102]. While prior evidence suggests that higher recall can enhance generation quality [51, 69], there has been no consensus on the optimal recall threshold. Figure 7b illustrates the impact of varying the percentage of scanned database vectors, ranging from 0.01% to 1% (with 0.1% as the default), on the proportion of time spent on retrieval across different model sizes. For all models, increasing the scanned database vectors significantly amplifies the proportion of time spent on retrieval, highlighting the substantial variability in retrieval performance across RAG configurations.

5.2 Case II: Long-Context Sequence Processing

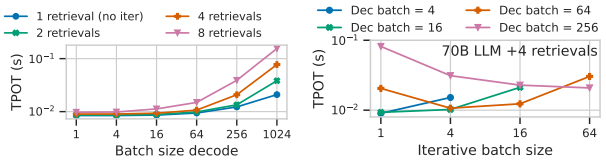
As shown in Table 3, we evaluate context lengths ranging from 100K to 10M tokens, resulting in a database of 1K to 100K vectors, with each chunk sized at 128 tokens and small overlaps between chunks. We use a sentence transformer model with 120 M parameters [96] to encode the passages, generating 768-dimensional embeddings, as relatively compact models are sufficient to achieve high retrieval quality [96]. Instead of ANN search, we use brute-force kNN search due to the high indexing costs associated with newly generated embeddings.

Takeaways: In contrast to Case I, retrieval performance plays a minimal role here. Instead, the database vector encoding process emerges as the bottleneck, even with a small encoder model, due to the significantly longer context the encoder must process compared to the generative LLM.



(a) QPS/Chip 70B (b) Breakdown 70B

Figure 8: RAG performance for long-context processing.



(a) Retrieval frequency (b) Prefix-retrieval batch size

Figure 9: RAG performance with iterative retrievals.

Sensitivity to context length. Figure 8 presents performance trends when the input context length scales from 100K to 10M tokens for the 70B model. "No long context" line represents the standard prompt length of a 512-token prefix. As the context length increases, RAG performance gradually degrades due to the increasing cost of context encoding, even though retrieval enables prompt truncation for the generative LLM. This happens due to database encoding becoming the bottleneck (Figure 8), especially at longer context lengths (>1M). Notably, encoding time scales with context length, despite the relatively small encoder applied (120M parameters), due to the sheer volume of data processed. Therefore, caching the generated embedding for potential reuse can significantly reduce computation with minimal cost. For instance, caching 10K 768-d database vectors in FP16 format (for 1M tokens) requires only 15 MB of CPU memory or storage. The retrieval time is minimal (0.01% to 0.4% of the end-to-end latency) even when using brute-force search due to small database (1K-100K vectors vs 64B in other cases).

RAG vs. long-context LLMs. Despite the high encoding cost, RAG is significantly more efficient than processing the entire long context as a prompt (long-context LLM). For instance, with a 1M-token context and a 70B LLM, RAG reduces the required prefix length to 512 tokens, achieving a speedup of 2852.6× in TTFT and 6633.9× in QPS/Chip. This is even considering an efficient long-context LLM applying global attention to all tokens in only one out of every four layers, while the rest layers only apply local attention to the last 128 tokens. This cost efficiency arises for two primary reasons: (I) In long-context RAG, the database encoder, typically a small model (e.g., 120M parameters), performs the encoding. This is much less computationally intensive compared to the LLM-only system with billions of parameters, which would require significantly more FLOPs if fed the entire context. (II) Long-context LLMs require key-value caches for every token, consuming substantial XPU memory (i.e. cost). In contrast, RAG significantly reduces prompt lengths, saving XPU memory. This distinction enables RAG to handle larger batch sizes during generation, increasing QPS/Chip.

5.3 Case III: Iterative Retrievals + Prefix

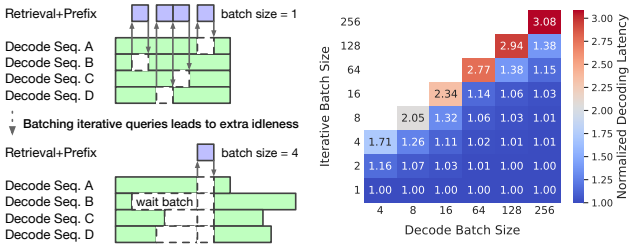
Building on Case I with hyperscale retrieval, this study adopts an iterative retrieval setup, allowing for 2, 4, or 8 retrievals per sequence generation process. Each retrieval is triggered at random intervals during the 256-token decoding process, with retrievals uniformly distributed across token positions.

Takeaways: Batch sizes for iterative retrievals must be carefully selected, as they significantly impact TPOT latency. Larger batch sizes improve retrieval and prefix throughput but may stall decoding.

Sensitivity to retrieval frequency. Figure 9a examines the impact of different retrieval frequencies (1-8 per sequence) on TPOT latency as the decode batch size increases from 1 to 1024, as QPS/Chip shows similar trends as multi-query retrieval in Case I. The results indicate that TPOT latency increases with both retrieval frequency and the decode batch size. At smaller decode batch sizes (one, four, and 16), the TPOT latency differences between retrieval frequencies are relatively minor. This is because, at these lower batch sizes, the decode step remains the dominant factor in TPOT latency, contributing approximately 60%-80% of the latency, while the effect of additional retrievals remains limited. At higher batch sizes, however, the decode process achieves higher QPS/Chip, reducing its share of the overall TPOT latency. This shift in bottleneck exposes the impact of retrieval frequency, as retrievals become the primary contributor to latency. Consequently, at larger batch sizes, the latency gap across different retrieval frequencies widens, making the increased time required for multiple retrievals more pronounced.

Sensitivity to iterative retrieval batch size. In Figure 9b, we observe the nuanced interplay between decode batch size, iterative retrieval-prefix batch size and TPOT latency for a 70B model processing four retrievals per sequence. At smaller decoding batch sizes (4 and 16), increasing the iterative retrieval batch size results in a noticeable increase in latency. This is due to the inherent challenge in finding enough retrieval requests to batch within an active set of decoding sequences over a given time interval, introducing stalls. For decode batch sizes of 256, the relationship reverses. As the iterative retrieval batch size increases, latency decreases. Here, the abundance of active decode sequences allow the system to batch retrieval requests more rapidly, enabling improved performance. The decode batch size of 64 presents a particularly intriguing case: it reaches its lowest TPOT at retrieval batch size of four. This optimal point represents a balance where idle time is minimized and the batching of retrieval requests is most efficient. However, beyond this threshold, latency begins to climb again as it becomes progressively harder to amass a sufficient number of retrieval requests for efficient batching. This behavior illustrates the delicate balance in RAG system performance when trying to balance retrieval performance and decoding efficiency.

Figure 10 further illustrates the phenomenon of decoding slowdown caused by idleness. Figure 10a visualizes the batching process, while the heatmap (Figure 10b) shows normalized decoding latency (compared to no retrieval) as a function of the decode batch size (x-axis) and iterative retrieval batch size (y-axis). In this evaluation, the retrieval and prefix stages are assumed to have zero latency, isolating the slowdown to the batching-induced waiting time. The results show that the effective latency is highly sensitive to the ratio of decode batch size to iterative retrieval batch size. When these batch sizes are similar (e.g., both set to 64), the normalized



(a) Wait for query batching (b) Performance degrade

Figure 10: Decode idleness due to batched iterative queries.

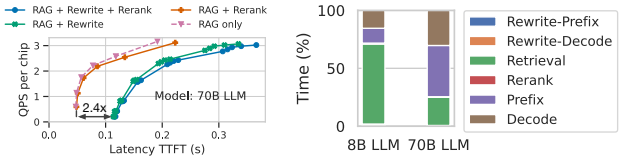


Figure 11: RAG performance with rewriter and reranker.

decoding latency reaches up to 2.77×. This increase occurs because one of the requests may generate a larger number of tokens before the next retrieval, resulting in idleness becomes a dominant factor. For smaller ratios (e.g., decode batch size 64 and retrieval batch size up to 16), latency increases more gradually, indicating a more balanced workload with minimal idleness. This observation aligns with Figure 9b, where, for a decode batch size of 64, increasing the iterative retrieval batch size from 16 (1.14× normalized latency due to idleness) to 64 (2.77× normalized latency due to idleness) causes a significant increase in TPOT latency. In summary, the results suggest that (a) when there is a large pool of XPUs that allows for large decoding batches, one can choose the iterative batch size that saturates database throughput, however, (b) with a smaller pool of XPUs and smaller decoding batch sizes, the optimal decoding batch size may actually be lower than the one that fully saturates the database.

5.4 Case IV: Query Rewriter and reranker

In this setup, we extend Case I by integrating an 8B query rewriter model [29] and a 120M reranker [96]. The rewriter processes a 32-token question and generates a rephrased question of the same length, while the reranker evaluates 16 nearest passages, each containing 100 tokens, and returns the top five nearest neighbors.

Takeaways: While the reranker has negligible impact on overall RAG performance, the query rewriter can significantly increase TTFT latency due to its autoregressive nature.

System performance with rewriter and reranker models. Figure 11 (left) presents the performance of various RAG configurations with or without rewriter and reranker. The results indicate that QPS/Chip remains largely unaffected by the addition of the rewriting and reranking modules. This is further validated from Figure 11 which shows that negligible time is spent in rewriter and reranker stages. However, the TTFT latency increases significantly (2.4×) when the rewriter is included, due to its autoregressive generation nature, while reranking has minimal impact on TTFT. This highlights the importance of considering an application's latency tolerance when integrating the rewriter model.

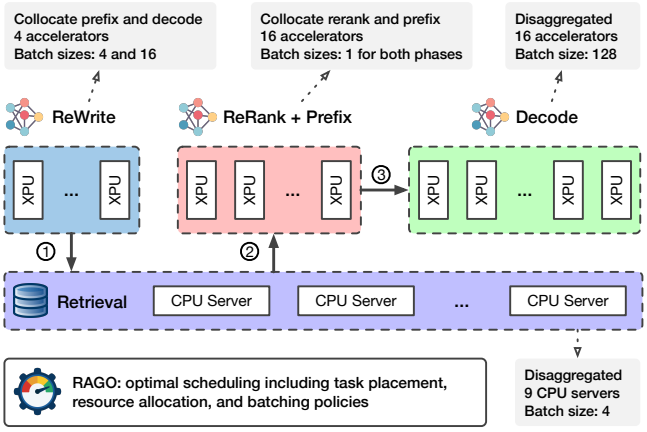


Figure 12: An example of RAGO optimizing placement, allocation, and batching policies for efficient RAG serving.

6 RAGO: Systematic RAG Serving Optimization

Given the heterogeneous components and high workload variance across RAG (§5), one-size-fits-all systems are inherently inadequate for achieving optimal serving efficiency. To overcome this challenge, we introduce **RAGO**, a systematic framework to design and optimize RAG serving systems across diverse configurations. RAGO determines an optimized scheduling policy tailored to a specific RAGSchema and a defined performance target. The following sections expound on the components of RAGO and its overall design.

6.1 RAGO Scheduling Decisions

Each scheduling solution comprises three pivotal system decisions: *task placement, resource allocation, and batching policy.* Figure 12 illustrates an example how these decisions come together to optimize a RAG serving pipeline under the constraint of 36 XPUs. In this example, RAGO adopts a hybrid collocation-disaggregation task placement strategy. Specifically, the pipeline is organized into two collocated subsystems: (1) the rewrite-prefix and rewrite-decode phases; and (2) the rerank and prefix phases of response generation. This organization ensures that tightly coupled tasks are efficiently grouped. Resource allocation is tailored to the computational demands of each subsystem. For instance, the query rewriter is assigned four XPUs, while the decoding phase, requiring significantly higher computational power, is allocated 16 XPUs. To further enhance efficiency, RAGO assigns batching policies customized to the characteristics of each phase. For example, the rerank and prefix phases prioritize low-latency processing with a batch size of one, whereas the decoding phase operates with a much larger batch size of 128 to maximize throughput. Below, we formally describe each system scheduling decision in RAGO, deferring how to search for optimal schedules to §6.2.

[I] Task placement. Recent LLM serving systems [89, 132] advocate for disaggregating the prefix and decode phases (§2), as these phases exhibit distinct workload characteristics —compute-bound vs. memory-bound— and impact TTFT versus TPOT. However, given the multiple components in a RAG pipeline (Figure 3), a natural question arises: *should RAG systems adhere to the convention of fully disaggregated designs common in LLM-only systems?*

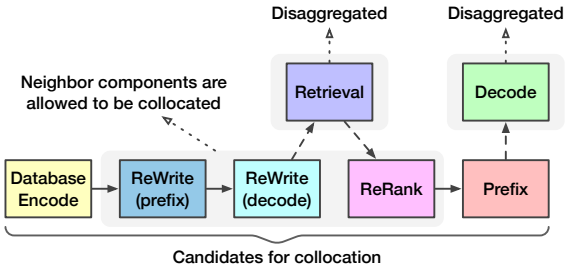


Figure 13: RAGO allows collocation of neighbor models.

While prefix-decode disaggregation often proves beneficial (§2), RAG pipelines may benefit more from collocation or hybrid strategies — where some model components are collocated on the same set of XPUs while others are disaggregated over different XPUs — particularly for components leading up to the prefix phase. First, several components in the pipeline — such as the database encoder, reranker, and the prefix phases of both the query rewriter and the main LLM — share a similar profile of high computational intensity, and thus time-multiplexing these components on the same set of XPUs can inherently mitigate workload imbalances among them. Second, components up to the prefix phase directly influence TTFT latency: while a fully disaggregated design, constrained by limited accelerator resources per stage, can prolong TTFT, collocation mitigates this by allowing all components to share the available resources, thereby reducing latency.

That said, the decision between collocation and disaggregation depends on the specific characteristics of the RAG pipeline. For instance, the decoding phase of the query rewriter is autoregressive, and scales pooly with small batch sizes even with additional XPUs [62, 120]. Thus, collocating it with the prefix phase across many chips risks underutilizing hardware resources, as analyzed in §7. To address these challenges, RAGO supports hybrid collocation-disaggregation task placement policies to balance flexibility and performance. Specifically, we make the following assumptions regarding placement policies. Firstly, the main LLM's prefix and decode phases remain disaggregated, consistent with the strategies in [89, 132]; Secondly, retrieval is always treated as a disaggregated task, as it operates on CPUs rather than XPUs. Finally, neighboring phases up to the prefix can be collocated (Figure 13). Collocation is restricted to consecutive neighbors to avoid excessively complicating the search space.

[II] Resource allocation. After determining task placement, RAGO assigns resources to each pipeline phase based on its computational and memory requirements. For collocated inference phases, this involves selecting the appropriate number of accelerators to ensure efficient execution. Similarly, for retrieval operations, RAGO determines the number of CPU servers required to meet workload demands. The framework balances throughput requirements and latency constraints to ensure optimal performance. Additionally, RAGO ensures that each component has sufficient accelerator or CPU memory capacity to store the required models or database segments while meeting the specified performance targets.

[III] Batching policy. Given a batch of incoming requests, RAGO enables each stage of the pipeline to adopt varying batch sizes, offering flexibility to balance latency and throughput at each stage. Upon receiving a burst of user requests, one can either use the same

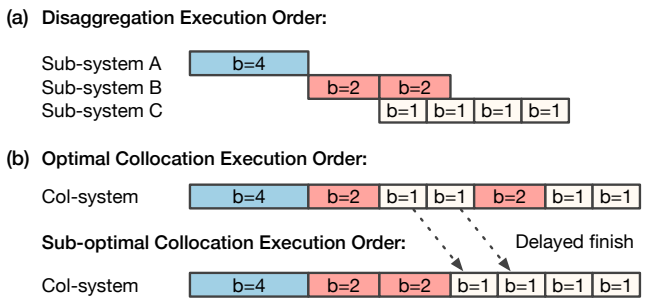


Figure 14: Execution order of batched requests until prefix.

batch size for all stages before decoding or divide the requests into micro-batches with the same or different batch sizes. For the decode stage, RAGO leverage continuous batching [62, 120] to use larger batch sizes than individual requests, thereby improving throughput, as we evaluate in §7. Moreover, in the case of iterative retrievals (§5.3), RAGO allows distinct batch sizes for the *initial* retrieval/prefix pair and the *subsequent* decoder-initiated retrieval/prefix iterations. This differentiation is crucial because the initial retrieval and prefix phases directly affect TTFT, while the iterative ones primarily impact TPOT during token generation (see §5.3).

Once batch sizes are determined, RAGO organizes their execution order to maximize efficiency based on the task placement strategy. Here, we discuss the order of stages up to prefix, as the generative LLM decode always apply continuous batching [62, 120]. In a fully disaggregated design (Figure 14(a)), execution is straightforward. As soon as (1) sufficient inputs arrive for a subsystem and (2) the subsystem completes its previous batch, it processes the new batch and forwards the output to the next subsystem. On the other hand, Figure 14(b) shows the execution order of the collocated design. For simplicity, we use time-multiplexing strategy and leave more complex strategies such as simultaneous execution as future work. In time-multiplexed designs, the throughput of the collocated system is fixed once batch sizes are set for each stage. In such cases, a stage begins execution as soon as it accumulates enough inputs. As illustrated in Figure 14, the optimal execution order prioritizes completing the final stage (b=1) early over processing another round of the second stage (b=2), thereby minimizing the average completion time of the final stage. If a retrieval operation is required between collocated stages (e.g., between the rewrite and prefix stages), the system pauses until the retrieval phase is complete before resuming the next collocated model inference phase.

6.2 Searching for Optimal Scheduling Policies

Given a RAGSchema and hardware resource constraints, RAGO performs an exhaustive search across potentially millions of schedules to identify Pareto frontier for key performance metrics such as TTFT and QPS/Chip. Users can define the search granularity to constrain the search space, for example, by limiting the search to accelerator numbers or batch sizes that are powers of two.

As shown in Algorithm1, RAGO search for optimal schedules in three main steps. First, it performs performance profiling by evaluating each RAG component individually (e.g., retrieval, prefix, etc.) under varying resource allocations and batch sizes. This evaluation relies on the calibrated analytical model introduced in §4. Next, it proceeds to schedule generation, where it explores *all* possible RAG schedules by considering (a) collocation strategies, (b) resource allocation strategies within the overall resource budget, and (c) batching strategies for each component. Finally, RAGO calculates the end-to-end performance for each schedule and identifying the Pareto frontier along with the corresponding schedule configurations.

Algorithm 1: Exhaustive Search for Optimal Scheduling

Input : RAGSchema parameters, resource constraints RC
Output : Performance Pareto frontier and schedules P_{RAG}
Step 1: Performance Profiling Per RAG Stage
Initialize $P_{\text{stage}} = \varnothing$ `// Performance per stage`
foreach *stage st in* RAGSchema **do**
  $P_{\text{stage}}[st] = \varnothing$
  foreach *resource r where* $r < RC$ **do**
    $P_{\text{stage}}[st] \leftarrow P_{\text{stage}}[st] \cup \text{Perf}(st, r)$
  $P_{\text{stage}}[st] \leftarrow \text{getPareto}(P_{\text{stage}}[st])$
Step 2: Schedule Generation
$SC_{\text{p}} \leftarrow \text{getPlacementOptions}(P_{\text{stage}}, RC)$
$SC_{\text{a}} \leftarrow \text{getAllocationOptions}(P_{\text{stage}}, RC, SC_{\text{p}})$
$SC_{\text{b}} \leftarrow \text{getBatchingOptions}(P_{\text{stage}}, RC)$
Step 3: End-to-end RAG Performance Evaluation
Initialize
 $P_{\text{RAG}} = \varnothing$ `// Performance of various schedules`
foreach *schedule* $sc_{e2e} \in SC_p \times SC_a \times SC_b$ **do**
  $P_{\text{RAG}} \leftarrow P_{\text{RAG}} \cup \text{assemblePerf}(sc_{\text{e2e}}, P_{\text{stage}})$
$P_{\text{RAG}} \leftarrow \text{getPareto}(P_{\text{RAG}})$
return P_{RAG} `// Pareto frontier and schedules`

7 Evaluation

We evaluate the effectiveness of RAGO by revisiting the four RAG case studies in §5. We begin with an analysis of the performance overview across all scheduling policies, followed by a detailed examination of each scheduling decision: placement, allocation, and batching.

Evaluation methodology. For evaluating placement and resource allocation decisions, we focus on case study II (C-II) —long-context sequence— and case study IV (C-IV) —RAG with rewriter and reranker. We select these case studies because of their additional model components, which distinguish them from LLM-only systems. For micro-batching policy evaluations under bursts of user requests, we include case study I (C-I) with hyperscale retrieval, alongside C-II and C-IV. We exclude case study III (C-III), which focuses on iterative retrievals during decoding, as it was evaluated in details in §5.3.

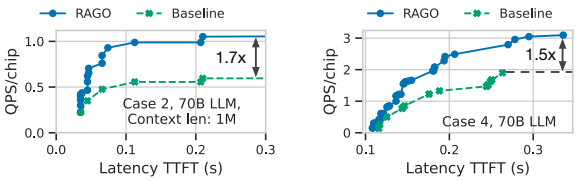
7.1 Overall Performance

Baseline system. Our baseline is an extension of LLM-only systems, where additional RAG components are collocated with the prefix system of the generative LLM. Rather than arbitrarily assigning chips to prefix and decode, we carefully tune the ratio based on their time consumption. In this tuned baseline, the prefix and decode stages are allocated in a 1:1 chip ratio, reflecting their similar time requirements in the pipeline (1.2∼1.4:1 across the 8B and 70B models).

Impact of scheduling policies on QPS/Chip. Figure 15a illustrates the Pareto performance comparison between RAGO and the

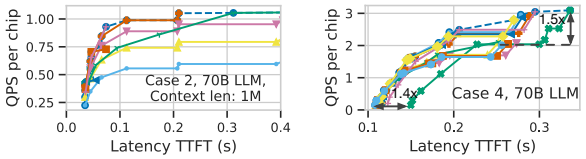
Table 4: Comparison of RAGO and baseline system schedules (placement, allocation, and batching strategies) in Case II.

Schedules	Performance		Batch Sizes				Num XPUs			
	TTFT (s)	QPS/Chip	Encode	Retrieval	Prefix	Decode	Encode	Prefix	Decode	Total
RAGO (Max QPS/Chip)	2.47	1.08	2	2	128	1024	64	16	16	96
RAGO (Min TTFT)	0.03	0.22	1	1	1	128	64 (col)	64 (col)	64	128
Baseline (Max QPS/Chip)	1.54	0.65	2	2	128	256	64 (col)	64 (col)	64	128
Baseline (Min TTFT)	0.03	0.22	1	1	1	128	64 (col)	64 (col)	64	128



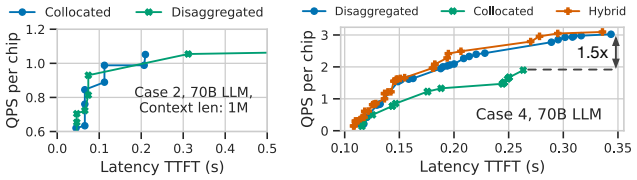
(a) Case II (b) Case IV

Figure 15: RAGO versus LLM-only system extension.



(a) Case II (b) Case IV

Figure 16: Performance Pareto across multiple placement and allocation plans in case 2 and 4.



(a) Case II (b) Case IV

Figure 17: Comparison of task placement plans.

baseline in terms of QPS/Chip across two distinct RAG case studies. In C-II, RAGO achieves a 1.7× improvement in maximum QPS/Chip over the baseline. This speedup underscores the inefficiencies of the baseline approach, particularly in handling the encoding stage for long-context sequences. The encoder, while smaller than the generative LLM, becomes a critical bottleneck as context lengths grow. Specifically, in the baseline configuration, encoding is collocated with the prefix stage, leading to resource imbalance: decoding XPUs (50% of all XPUs) remain idle, while encode-prefix XPUs are overloaded. This imbalance can theoretically reduce QPS/Chip by up to 2.0× in the baseline, which aligns with our observed reduction of 1.94× for a large 10M-token context, though this specific data point is not plotted. On the other hand, RAGO achieves high QPS/Chip by allocating 64 out of the 96 XPUs to encoding (Table 4), reflecting the high time consumption of this stage.

A similar inefficiency of the baseline is observed in C-IV (Figure 15b), where the rewriter and reranker models, despite their relatively small size (8B and 120M), significantly impact throughput in the baseline system. This QPS drop can be attributed to two primary factors. First, collocating rewriter-decode stage and the prefix stage of the main generative LLM leads to XPU under-utilization due to the low computational intensity of the autoregressive decoding stage, particularly when handling small batch sizes. Second, retrieval operations are introduced between the rewriting and prefix stages add wait times for retrieval results (e.g., 10 ms with a batch size of one given 32 host servers), further reducing throughput. In contrast, RAGO demonstrates its ability to mitigate these bottlenecks through optimized task placement, resource allocation, and batching strategies. These results highlight the importance of disaggregating smaller pipeline stages and balancing resource distribution to unlock the full throughput potential of RAG systems.

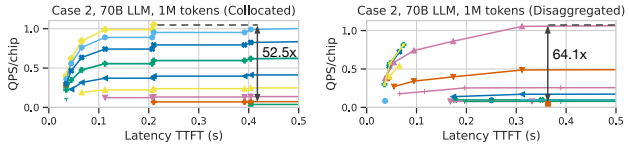
Pareto composition analysis. Figure 16a and Figure 16b reveal how diverse placement and allocation plans contribute to the overall Pareto frontier. The dashed lines represent the global Pareto frontier, while each solid line corresponds to the Pareto frontier of a specific combination of placement and allocation strategies, with each point on a line representing a batching policy. We omit the legend for each plan, as providing detailed descriptions for each would take up excessive space, but will explain some of these plans below. The overall Pareto frontier is constructed from multiple distinct plans, each embodying a unique trade-off between TTFT and QPS/Chip.This diversity underscores the importance of tailoring placement and allocation strategies to the specific performance priorities of a deployment. For instance, as shown in Figure 16b, selecting the most throughput-optimized plan results in a trade-off, with TTFT approximately 40% higher compared to the most latency-optimized plan, while achieving 1.5× QPS/Chip. This is because the throughput-optimized plan allocates only one chip to the query rewriter, given its minimal contribution to the end-to-end generation latency, as analyzed in §5.4. In contrast, the latency-optimized plan allocates 32 chips to the query rewriter, resulting in low resource utilization since a significant number of chips are assigned to this non-bottleneck stage. These findings emphasize that there is no one-size-fits-all strategy. Instead, the optimal placement and allocation plans must be aligned with the operational objectives, whether minimizing latency, maximizing throughput, or achieving a balance between the two.

7.2 Scheduling Policy Sensitivity Analysis

We now delve into a detailed analysis of the performance implication of each scheduling decision.

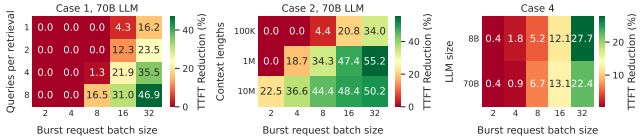
Task placement sensitivity. Figure 17 compares the impact of different task placement policies on system performance across C-II and C-IV. Each line in Figure 17a and Figure 17b represents the Pareto frontier for a specific placement strategy, illustrating the relationship between QPS/Chip and TTFT latency under these policies.

In C-II (Figure 17a), task placement sensitivity is minimal. Both collocated and disaggregated strategies yield comparable



(a) Collocated (b) Disaggregated

Figure 18: Comparison of resource allocation plans (case II).



(a) Case I (b) Case II (c) Case IV

Figure 19: TTFT latency reduction by micro-batching.

performance, as the encoder and prefix stages are computationally intensive. Whether these stages are time-multiplexed (collocated) or spatially multiplexed (disaggregated), performance remains consistent (only 2% difference in max QPS/Chip) as long as the accelerator ratio between stages is appropriately chosen. This demonstrates that task placement decisions in this case have little effect on system efficiency, provided resources are balanced effectively.

In contrast, C-IV (Figure 17b) shows a more pronounced sensitivity to placement policies. Here, a hybrid placement strategy —where some model components are collocated on the same set of XPUs while others are disaggregated over different XPUs— slightly outperforms the fully disaggregated approach and significantly surpasses the collocated plan, achieving up to a 1.5× improvement in QPS/Chip. The key advantage of the hybrid and disaggregated strategies lies in their ability to mitigate the underutilization of prefix chips, which occurs when the rewriter model is collocated with the prefix stage. By separating the rewriter model from the prefix system, these strategies prevent resource bottlenecks and enable optimal throughput.

Resource allocation sensitivity. Figure 18 shows the high sensitivity of performance to resource allocation strategies regardless of whether collocated or disaggregated placement plans are used. Each curve represents the Pareto frontier of a specific resource allocation strategy in Case II. Due to space constraints, the legend for each plan is omitted, with explanations for selected plans provided below. For collocated plans, the maximum QPS/chip can vary by up to 52.5× if insufficient resources are allocated to high-workload stages, when other stages have surplus capacity. For example, in the collocated plan (Figure 18a), imbalanced resource distribution across the encoder and prefix stages leads to underutilization of available accelerators, limiting throughput. This effect may amplify to 64.1× QPS/chip difference for disaggregated plans, as disaggregated stages rely heavily on precise balancing to maximize performance. Tailoring resource distribution to the specific demands of each stage is essential for optimizing both latency and throughput in RAG systems.

Impact of micro-batching on TTFT latency. The effectiveness of micro-batching depends on the throughput sensitivity to batch size at each pipeline stage. When a burst of user requests arrives, they can be divided into micro-batches across the stages prior to decoding. Figure 19 compares the impact of micro-batching on TTFT reduction across the case studies. For C-II (Figure 19b), micro-batching is effective even with a small batch size of two, reducing TTFT by 22%. This is because both the encoding and prefix stages are computationally intensive, achieving reasonable throughput even with smaller batch sizes. With larger batches of 32, the TTFT reduction increases further to 55% for 1M tokens. In C-I (Figure 19a), micro-batching only becomes effective with larger batch sizes, such as eight or 16. This inefficiency at smaller batch sizes arises from the vector search system, where reducing the query batch size below 16 fails to improve latency. However, with batch sizes increasing to 32, micro-batching still achieves a significant latency reduction of 46% for eight queries per vector. For C-IV (Figure 19c), TTFT reduction is moderate, with a maximum improvement of approximately 25% at a batch size of 32. This modest improvement is primarily due to the query rewriter decoding stage, which exhibits little latency reduction with smaller sequence batch sizes.

8 Additional Related Work

RAG performance optimization. As an emerging research area, RAG performance optimization remains underexplored, with existing studies targeting specific configurations. Chameleon[50] integrates both retrieval and LLM accelerators within RAG systems. Its retrieval accelerator is tailored for large-scale, product-quantization-based vector search, making it particularly beneficial in hyper-scale retrieval scenarios, where retrieval constitutes a major bottleneck (§5.1). PipeRAG[51] and RaLMSpec[128] address decoding stalls in iterative retrievals (§5.3) through data prefetching — PipeRAG employs approximate prefetching, whereas RaLMSpec incorporates an additional asynchronous verification step. Additionally, since RAG introduces increased prompt length and associated computation costs (§5.1), studies such as CacheBlend[119] and RAGCache [53] propose caching the KV-cache of database documents, a method that is particularly effective when the prompt-to-generation cost ratio is high. If these techniques are adopted, the workload distribution within a RAG system evaluated by RAGO is expected to shift. For example, retrieval acceleration [50] will shift the workload toward being more inference-bound; pre-computing KV cache of retrieved documents [53, 119] will increase the importance of retrieval and decoding performance; supporting data prefetching in iterative retrievals through [51, 128] will reduce decoding engine idleness during retrieval operations.

LLM and retrieval optimization. Extensive research has been devoted to optimizing LLM systems and their underlying hardware [19, 62, 66, 72, 89, 91, 100, 120, 123, 126, 131, 132]. Similarly, significant efforts have focused on enhancing retrieval efficiency, a fundamental topic in databases [80, 81, 114], which is essential not only for RAG but also for recommender systems [27, 104], scientific applications [18, 115], and video matching [34, 98, 114]. These efforts include both algorithm- and system-level optimizations [25, 44, 45, 83, 85, 87, 117, 127] as well as modern hardware supports for various retrieval algorithms, spanning product-quantization-based ANN algorithms [49, 54, 68, 77] and graph-based approaches [36, 43, 48, 124, 129]. However, the complexity and heterogeneity of RAG pipelines far exceed those of LLM-only or retrieval-only systems, highlighting the need for RAGO to perform system-level RAG scheduling.

9 Conclusion and Future Outlook

This work represents an early exploration of RAG through a systems lens, establishing a foundation for this rapidly evolving field. Our contribution, RAGSchema, provides a structured abstraction for RAG serving, facilitating systematic workload characterization and bridging the gap between algorithms and system design. Leveraging RAGSchema, we proposed RAGO, a system optimization framework that delivers up to a 2× improvement in QPS per chip and a 55% reduction in TTFT compared to a strong baseline.

As the field advances, our characterization results can guide the design of future RAG systems and hardware. For instance, our findings highlight that retrieval can become a bottleneck in certain RAG paradigms, particularly at hyperscale, underscoring the need for further retrieval optimizations. Moreover, with multiple model components in RAG pipelines, efficient support for collocated models on accelerators will be increasingly critical. Finally, scaling to extremely complicated RAG and agentic AI systems introduces challenges related to a broader optimization search space and additional efficiency metrics, such as energy and cost efficiency. We leave these investigations to future work.

Acknowledgments

We extend our gratitude towards Cliff Young, David Culler, and Eugene Le for reviewing the paper and providing insightful feedback. We also thank the extended team at Google DeepMind and System Research@Google who enabled and supported this research direction.

References

[1] 2021. TPU v4. https://cloud.google.com/tpu/docs/v4.
[2] 2023. TPU v5e. https://cloud.google.com/tpu/docs/v5e.
[3] 2023. TPU v5p. https://cloud.google.com/tpu/docs/v5p.
[4] 2025. Advanced RAG Techniques: Elevating Your Retrieval-Augmented Generation Systems. https://github.com/NirDiamant/RAG_Techniques.
[5] 2025. ANN-Benchmarks: A Benchmarking Environment for Approximate Nearest Neighbor Algorithms Search. https://ann-benchmarks.com/.
[6] 2025. AWS Inferentia. https://aws.amazon.com/ai/machine-learning/inferentia/.
[7] 2025. Faiss. https://github.com/facebookresearch/faiss/.
[8] 2025. NotebookLM: Note Taking and Research Assistant Powered by AI. https://notebooklm.google/.
[9] 2025. OpenAI ChatGPT. https://chat.openai.com/.
[10] 2025. ScaNN: Scalable Nearest Neighbors. https://github.com/google-research/google-research/blob/master/scann.
[11] 2025. ShareGPT: Share your ChatGPT conversations. https://sharegpt.com/.
[12] 2025. SIFT ANNS dataset. http://corpus-texmex.irisa.fr/
[13] Ritvik Aggarwal Ishneet Sukhvinder Singh Ibrahim Allahverdiyev, Muhammad Taha, Aslihan Akalin, and Kevin Zhu. 2024. ChunkRAG: Novel LLM-Chunk Filtering Method for RAG Systems. *arXiv preprint arXiv:2410.19572* (2024).
[14] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2016. Cache Locality is not Enough: High-performance Nearest Neighbor Search with Product Quantization Fast Scan. In *VLDB*.
[15] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511* (2023).
[16] Artem Babenko and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *CVPR*.
[17] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. 2016. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. *arXiv preprint arXiv:1611.09268* (2016).
[18] Dávid Bajusz, Anita Rácz, and Károly Héberger. 2015. Why is Tanimoto Index an Appropriate Choice for Fingerprint-Based Similarity Calculations? *Journal of Cheminformatics* (2015).
[19] Jehyeon Bang, Yujeong Choi, Myeongwoo Kim, Yongdeok Kim, and Minsoo Rhu. 2023. vTrain: A Simulation Framework for Evaluating Cost-effective and Compute-optimal Large Language Model Training. *arXiv preprint arXiv:2312.12391* (2023).
[20] Iz Beltagy, Kyle Lo, and Arman Cohan. 2019. SciBERT: A Pretrained Language Model for Scientific Text. *arXiv preprint arXiv:1903.10676* (2019).
[21] Maciej Besta, Ales Kubicek, Roman Niggli, Robert Gerstenberger, Lucas Weitzendorf, Mingyuan Chi, Patrick Iff, Joanna Gajda, Piotr Nyczyk, Jürgen Müller, et al. 2024. Multi-Head RAG: Solving Multi-Aspect Problems with LLMs. *arXiv preprint arXiv:2406.05085* (2024).
[22] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving Language Models by Retrieving From Trillions of Tokens. In *ICML*.
[23] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-shot Learners. In *NeurIPS*.
[24] Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. 2024. RQ-RAG: Learning to Refine Queries for Retrieval Augmented Generation. *arXiv preprint arXiv:2404.00610* (2024).
[25] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2111.08566* (2021).
[26] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. PaLM: Scaling Language Modeling with Pathways. *arXiv preprint arXiv:2204.02311* (2022).
[27] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *RecSys*.
[28] Databricks. 2024. RAG (Retrieval Augmented Generation) on Databricks.
[29] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
[30] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).
[31] Angela Fan, Yacine Jernite, Ethan Perez, David Grangier, Jason Weston, and Michael Auli. 2019. ELI5: Long form Question Answering. *arXiv preprint arXiv:1907.09190* (2019).
[32] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2017. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *arXiv preprint arXiv:1707.00143* (2017).
[33] Jianyang Gao and Cheng Long. 2023. High-dimensional Approximate Nearest Neighbor Search: With Reliable and Efficient Distance Comparison Operations. *Proceedings of the ACM on Management of Data* (2023).
[34] Vincent Garcia, Eric Debreuve, and Michel Barlaud. 2008. Fast K Nearest Neighbor Search using GPU. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*.
[35] Michael Glass, Gaetano Rossiello, Md Faisal Mahbub Chowdhury, Ankita Rajaram Naik, Pengshan Cai, and Alfio Gliozzo. 2022. Re2G: Retrieve, Rerank, Generate. *arXiv preprint arXiv:2207.06300* (2022).
[36] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik PA Lensch. 2022. GGNN: Graph-Based GPU Nearest Neighbor Search. *IEEE Transactions on Big Data* (2022).
[37] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *ICML*.
[38] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval Augmented Language Model Pre-training. In *ICML*.
[39] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. *arXiv preprint arXiv:2002.08909* (2020).
[40] Qijing Huang, Po-An Tsai, Joel S Emer, and Angshuman Parashar. 2024. Mind the Gap: Attainable Data Movement and Operational Intensity Bounds for Tensor Algorithms. In *ISCA*.
[41] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *NeurIPS*.
[42] Gautier Izacard and Edouard Grave. 2020. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *arXiv preprint arXiv:2007.01282* (2020).
[43] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *ATC*.
[44] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. *NeurIPS*.

[45] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2010).

[46] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* (2023).

[47] Wenqi Jiang, Zhenhao He, Shuai Zhang, Kai Zeng, Liang Feng, Jiansong Zhang, Tongxuan Liu, Yong Li, Jingren Zhou, Ce Zhang, et al. 2021. FleetRec: Large-scale Recommendation Inference on Hybrid GPU-FPGA Clusters. In *SIGKDD*.

[48] Wenqi Jiang, Hang Hu, Torsten Hoefler, and Gustavo Alonso. 2024. Accelerating Graph-based Vector Search via Delayed-Synchronization Traversal. *arXiv preprint arXiv:2406.12385* (2024).

[49] Wenqi Jiang, Shigang Li, Yu Zhu, Johannes de Fine Licht, Zhenhao He, Runbin Shi, Cedric Renggli, Shuai Zhang, Theodoros Rekatsinas, Torsten Hoefler, and Gustavo Alonso. 2023. Co-design Hardware and Algorithm for Vector Search. In *SC*.

[50] Wenqi Jiang, Marco Zeller, Roger Waleffe, Torsten Hoefler, and Gustavo Alonso. 2025. Chameleon: A Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models. *VLDB* (2025).

[51] Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. 2025. PipeRAG: Fast Retrieval-Augmented Generation via Algorithm-System Co-design. In *KDD*.

[52] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active Retrieval Augmented Generation. *arXiv preprint arXiv:2305.06983* (2023).

[53] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *arXiv preprint arXiv:2404.12457* (2024).

[54] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* (2019).

[55] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. *arXiv preprint arXiv:1705.03551* (2017).

[56] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *ISCA*.

[57] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. 2023. ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education. *Learning and Individual Differences* (2023).

[58] Liu Ke, Xuan Zhang, Benjamin Lee, G Edward Suh, and Hsien-Hsin S Lee. 2022. DisaggRec: Architecting Disaggregated Systems for Large-Scale Personalized Recommendation. *arXiv preprint arXiv:2212.00939* (2022).

[59] Urvashi Khandelwal, Angela Fan, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2020. Nearest Neighbor Machine Translation. *arXiv preprint arXiv:2010.00710* (2020).

[60] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2019. Generalization through Memorization: Nearest Neighbor Language Models. *arXiv preprint arXiv:1911.00172* (2019).

[61] Mojtaba Komeili, Kurt Shuster, and Jason Weston. 2021. Internet-Augmented Dialogue Generation. *arXiv preprint arXiv:2107.07566* (2021).

[62] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SIGOPS*.

[63] Angeliki Lazaridou, Adhi Kuncoro, Elena Gribovskaya, Devang Agrawal, Adam Liska, Tayfun Terzi, Mai Gimenez, Cyprien de Masson d'Autume, Tomas Kocisky, Sebastian Ruder, et al. 2021. Mind the Gap: Assessing Temporal Generalization in Neural Language Models. In *NeurIPS*.

[64] Jinhyuk Lee, Anthony Chen, Zhuyun Dai, Dheeru Dua, Devendra Singh Sachan, Michael Boratko, Yi Luan, Sébastien MR Arnold, Vincent Perot, Siddharth Dalmia, et al. 2024. Can Long-Context Language Models Subsume Retrieval, RAG, SQL, and More? *arXiv preprint arXiv:2406.13121* (2024).

[65] Jinhyuk Lee, Zhuyun Dai, Xiaoqi Ren, Blair Chen, Daniel Cer, Jeremy R Cole, Kai Hui, Michael Boratko, Rajvi Kapadia, Wen Ding, et al. 2024. Gecko: Versatile Text Embeddings Distilled From Large Language Models. *arXiv preprint arXiv:2403.20327* (2024).

[66] Jungi Lee, Wonbeom Lee, and Jaewoong Sim. 2024. Tender: Accelerating Large Language Models via Tensor Decomposition and Runtime Requantization. *arXiv preprint arXiv:2406.12930* (2024).

[67] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2020. BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining. *Bioinformatics* (2020).

[68] Yejin Lee, Hyunji Choi, Sunhong Min, Hyunseung Lee, Sangwon Beak, Dawoon Jeong, Jae W Lee, and Tae Jun Ham. 2022. ANNA: Specialized Architecture for Approximate Nearest Neighbor Search. In *HPCA*.

[69] Alexandria Leto, Cecilia Aguerrebere, Ishwar Bhati, Ted Willke, Mariano Tepper, and Vy Ai Vo. 2024. Toward Optimal Search and Retrieval for RAG. *arXiv preprint arXiv:2411.07396* (2024).

[70] Mike Lewis, Marjan Ghazvininejad, Gargi Ghosh, Armen Aghajanyan, Sida Wang, and Luke Zettlemoyer. 2020. Pre-training via Paraphrasing. In *NeurIPS*.

[71] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

[72] Jinhao Li, Jiaming Xu, Shan Huang, Yonghua Chen, Wen Li, Jun Liu, Yaoxiu Lian, Jiayi Pan, Li Ding, Hao Zhou, et al. 2024. Large Language Model Inference Acceleration: A Comprehensive Hardware Perspective. *arXiv preprint arXiv:2410.04466* (2024).

[73] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-Level Code Generation with AlphaCode. *Science* (2022).

[74] Zihao Li. 2023. The Dark Side of ChatGPT: Legal and Ethical Challenges From Stochastic Parrots and Hallucination. *arXiv preprint arXiv:2304.14347* (2023).

[75] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. 2024. Retrieval Augmented Generation or Long-context LLMs? A Comprehensive Study and Hybrid Approach. *arXiv preprint arXiv:2407.16833* (2024).

[76] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *NeurIPS*.

[77] Zihan Liu, Wentao Ni, Jingwen Leng, Yu Feng, Cong Guo, Quan Chen, Chao Li, Minyi Guo, and Yuhao Zhu. 2023. JUNO: Optimizing High-Dimensional Approximate Nearest Neighbour Search with Sparsity-Aware Algorithm and Ray-Tracing Core Mapping. *arXiv preprint arXiv:2312.01712* (2023).

[78] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: Hierarchical Graph Structure based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. *VLDB* (2021).

[79] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query Rewriting for Retrieval-Augmented Large Language Models. *arXiv preprint arXiv:2305.14283* (2023).

[80] Divya Mahajan, Joon Kyung Kim, Jacob Sacks, Adel Ardalan, Arun Kumar, and Hadi Esmaeilzadeh. 2018. In-RDBMS Hardware Acceleration of Advanced Analytics. *VLDB* (2018).

[81] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyan Feng, Simon Garcia De Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. 2019. DeepStore: In-Storage Acceleration for Intelligent Queries. In *MICRO*.

[82] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate Nearest Neighbor Algorithm based on Navigable Small World Graphs. *Information Systems* (2014).

[83] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2018).

[84] Meta. 2024. Build AI Knowledge Assistants over your Enterprise Data.

[85] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-Throughput Vector Similarity Search in Knowledge Graphs. *Proceedings of the ACM on Management of Data* (2023).

[86] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *SOSP*.

[87] James Jie Pan, Jianguo Wang, and Guoliang Li. 2023. Survey of Vector Database Management Systems. *arXiv preprint arXiv:2310.14021* (2023).

[88] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *ISPASS*.

[89] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. *arXiv preprint arXiv:2311.18677* (2023).

[90] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable Code Generation From Pre-trained Language Models. *arXiv preprint arXiv:2201.11227* (2022).

[91] Yubin Qin, Yang Wang, Zhiren Zhao, Xiaolong Yang, Yang Zhou, Shaojun Wei, Yang Hu, and Shouyi Yin. 2024. MECLA: Memory-Compute-Efficient LLM Accelerator with Scaling Sub-matrix Partition. In *ISCA*.

[92] Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. 2021. Scaling Language Models: Methods, Analysis & Insights From Training Gopher. *arXiv preprint arXiv:2112.11446* (2021).

[93] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *SC*.

[94] Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know What You Don't Know: Unanswerable Questions for SQuAD. *arXiv preprint arXiv:1806.03822* (2018).

[95] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-Context Retrieval-Augmented Language Models. *arXiv preprint arXiv:2302.00083* (2023).

[96] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv preprint arXiv:1908.10084* (2019).

[97] Michael Ruminer. 2024. Google's NotebookLM and RAG.

[98] Jie Shao, Zi Huang, Heng Tao Shen, Xiaofang Zhou, Ee-Peng Lim, and Yijun Li. 2008. Batch Nearest Neighbor Search for Video Retrieval. *IEEE Transactions on Multimedia* (2008).

[99] Rulin Shao, Jacqueline He, Akari Asai, Weijia Shi, Tim Dettmers, Sewon Min, Luke Zettlemoyer, and Pang Wei Koh. 2024. Scaling Retrieval-Based Language Models with a Trillion-Token Datastore. *arXiv preprint arXiv:2407.12854* (2024).

[100] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, et al. 2019. Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture. In *MICRO*.

[101] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[102] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, et al. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2205.03763* (2022).

[103] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. 2022. Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530B, A Large-Scale Generative Language Model. *arXiv preprint arXiv:2201.11990* (2022).

[104] Ján Suchal and Pavol Návrat. 2010. Full Text Search Engine as Scalable K-Nearest Neighbor Recommendation System. In *IFIP*.

[105] Philip Sun, Ruiqi Guo, and Sanjiv Kumar. 2023. Automating Nearest Neighbor Search Configuration with Constrained Optimization. *arXiv preprint arXiv:2301.01702* (2023).

[106] Philip Sun, David Simcha, Dave Dopson, Ruiqi Guo, and Sanjiv Kumar. 2024. SOAR: Improved Indexing for Approximate Nearest Neighbor Search. In *NeurIPS*.

[107] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A Large Language Model for Science. *arXiv preprint arXiv:2211.09085* (2022).

[108] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context. *arXiv preprint arXiv:2403.05530* (2024).

[109] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language Models in Medicine. *Nature medicine* (2023).

[110] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. *arXiv preprint arXiv:2212.10509* (2022).

[111] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *NeurIPS*.

[112] Boxin Wang, Wei Ping, Lawrence McAfee, Peng Xu, Bo Li, Mohammad Shoeybi, and Bryan Catanzaro. 2023. InstructRetro: Instruction Tuning post Retrieval-Augmented Pretraining. *arXiv preprint arXiv:2310.07713* (2023).

[113] Shuting Wang, Xin Xu, Mang Wang, Weipeng Chen, Yutao Zhu, and Zhicheng Dou. 2024. RichRAG: Crafting Rich Responses for Multi-faceted Queries in Retrieval-Augmented Generation. *arXiv preprint arXiv:2406.12566* (2024).

[114] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.

[115] Jonathan Woodbridge, Bobak Mortazavi, Alex AT Bui, and Majid Sarrafzadeh. 2016. Improving Biomedical Signal Search Results in Big Data Case-Based Reasoning Environments. *Pervasive and mobile computing* (2016).

[116] Frank F Xu, Uri Alon, and Graham Neubig. 2023. Why do Nearest Neighbor Language Models Work? *arXiv preprint arXiv:2301.02828* (2023).

[117] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, et al. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *SOSP*.

[118] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. 2024. CRAG–Comprehensive RAG Benchmark. *arXiv preprint arXiv:2406.04744* (2024).

[119] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2024. CacheBlend: Fast Large Language Model Serving with Cached Knowledge Fusion. *arXiv preprint arXiv:2405.16444* (2024).

[120] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI*.

[121] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. 2024. RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs. *arXiv preprint arXiv:2407.02485* (2024).

[122] Zhenrui Yue, Honglei Zhuang, Aijun Bai, Kai Hui, Rolf Jagerman, Hansi Zeng, Zhen Qin, Dong Wang, Xuanhui Wang, and Michael Bendersky. 2024. Inference Scaling for Long-Context Retrieval Augmented Generation. *arXiv preprint arXiv:2410.04343* (2024).

[123] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching. *arXiv preprint arXiv:2409.01141* (2024).

[124] Shulin Zeng, Zhenhua Zhu, Jun Liu, Haoyu Zhang, Guohao Dai, Zixuan Zhou, Shuangchen Li, Xuefei Ning, Yuan Xie, Huazhong Yang, et al. 2023. DF-GAS: a Distributed FPGA-as-a-Service Architecture towards Billion-Scale Graph-based Approximate Nearest Neighbor Search. In *MICRO*.

[125] Dan Zhang, Safeen Huda, Ebrahim Songhori, Kartik Prabhu, Quoc Le, Anna Goldie, and Azalia Mirhoseini. 2022. A Full-Stack Search Technique for Domain Optimized Deep Learning Accelerators. In *ASPLOS*.

[126] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. 2024. LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference. In *ISCA*.

[127] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *OSDI*.

[128] Zhihao Zhang, Alan Zhu, Lijie Yang, Yihua Xu, Lanting Li, Phitchaya Mangpo Phothilimthana, and Zhihao Jia. 2024. Accelerating Retrieval-Augmented Language Model Serving with Speculation. *arXiv preprint arXiv:2401.14021* (2024).

[129] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *ICDE*.

[130] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *VLDB* (2023).

[131] Youpeng Zhao, Di Wu, and Jun Wang. 2024. ALISA: Accelerating Large Language Model Inference via Sparsity-Aware KV Caching. *arXiv preprint arXiv:2403.17312* (2024).

[132] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *OSDI*.

[133] Chaoji Zuo and Dong Deng. 2023. ARKGraph: All-Range Approximate K-Nearest-Neighbor Graph. *VLDB* (2023).